



**UNIVERSIDAD CENTRAL DE VENEZUELA
FACULTAD DE CIENCIAS
ESCUELA DE COMPUTACIÓN
APLICACIONES CON TECNOLOGÍAS DE
INTERNET**

Laboratorio 5

Oriana Arellano C.I 28.309.661

Caracas, Junio de 2026

1. Análisis del uso de this en JavaScript

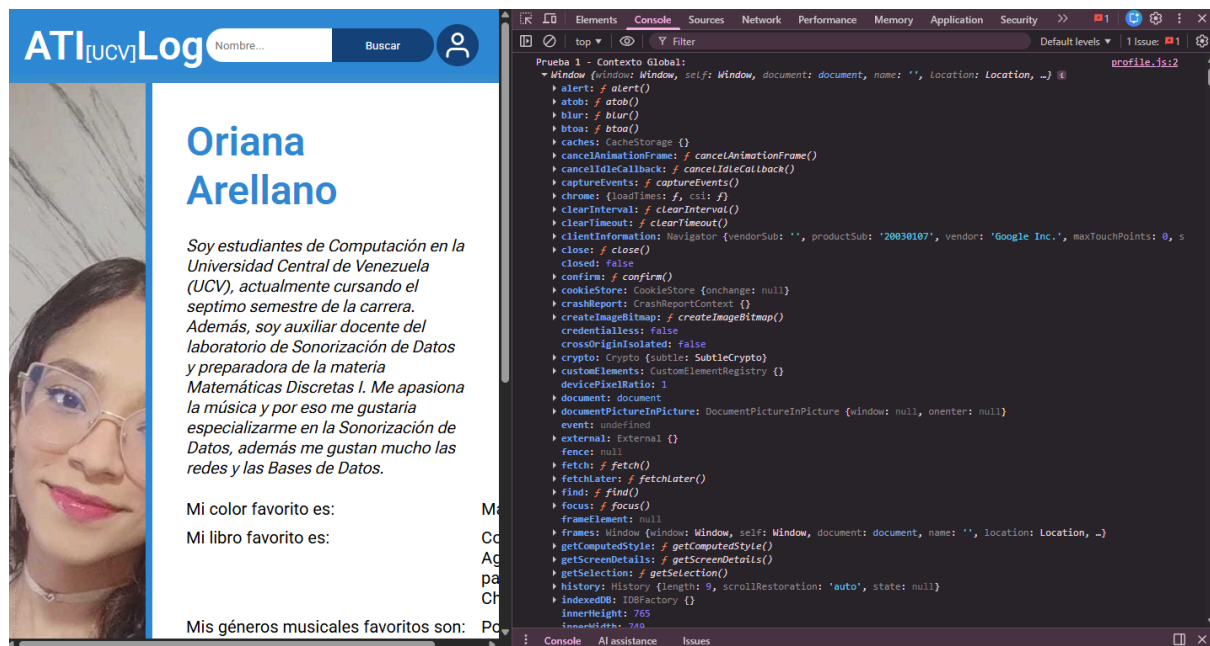


Imagen 1.a (1) - Captura de la consola mostrando el objeto Window

En este primer caso, el uso de this corresponde al Default Binding (Enlace por Defecto). Al invocar this en el contexto global dentro del navegador, el motor de JavaScript hace que apunte automáticamente al objeto global, que en este caso es el objeto Window. Esto ocurre porque no hay ningún contexto específico que lo esté envolviendo.

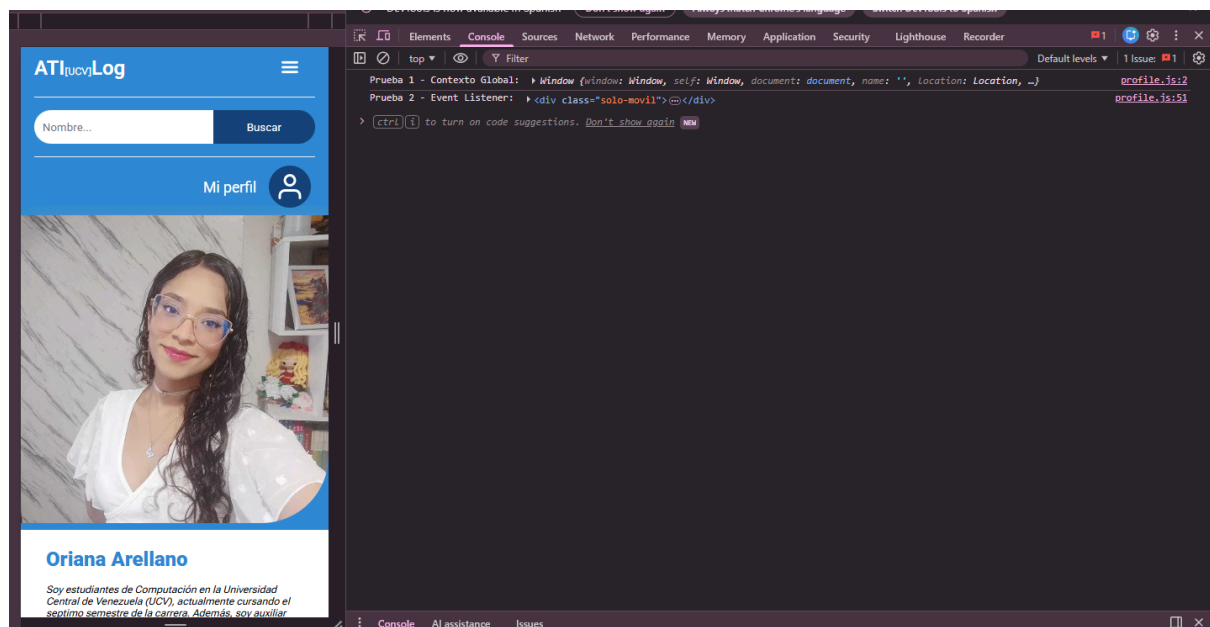


Imagen 1.a (2) - Captura de la consola mostrando la etiqueta HTML del botón móvil

En este caso, el uso de this corresponde a un Implicit Binding modificado por el DOM. Cuando utilizamos el método addEventListener con una función tradicional (function()), JavaScript enlaza automáticamente la palabra this al elemento del DOM que disparó el

evento. Por lo tanto, this dejó de ser Window y pasó a representar al nodo HTML del botón de menú que recibió el clic.

```
Prueba 3 - Objeto: ▶ {nombre: 'Oriana Arellano', mostrarThis: f} profile.js:104
El nombre es: Oriana Arellano profile.js:105
> ctrl i to turn on code suggestions. Don't show again NEW
```

Imagen 1.a (3) - Captura de la consola mostrando el objeto "perfilPrueba"

Este tercer caso demuestra el Implicit Binding (Enlace Implícito) estándar de JavaScript. Al declarar el método mostrarThis dentro del objeto constante perfilPrueba, la palabra this asume la identidad del objeto que está justo a la izquierda del punto al momento de ser ejecutado. Gracias a esto, this apunta directamente al objeto en sí, permitiendo acceder a sus propiedades internas como this.nombre.

2. Análisis del rendimiento del selector de CSS

Summary	Bottom-up	Call tree	Event log	Selector stats		
Elapsed (ms)	Invalidatio...	Match att...	Match cou...	% of slow...	Selector	Style Sheet
0.358	0	306	187	61.3	(Totals for all selectors)	
0.043	0	5	4	0.0	section[class] div:first-child img	Line 161:1
0.034	0	51	49	100.0	*	Line 3:1
0.032	0	4	2	0.0	section h1	Line 35:1,Line 10...
0.026	0	1	1	0.0	button	Line 65:1
0.023	0	1	1	0.0	input	Line 56:1
0.015	0	2	2	0.0	body	Line 13:1
0.015	0	3	1	0.0	footer p	Line 137:1,Line 3...
0.012	0	1	1	0.0	.ficha-usuario	Line 225:1,Line 2...
0.009	0	1	1	0.0	nav:first-child	Line 20:1
0.009	0	1	1	0.0	nav	Line 42:1,Line 31...
0.009	0	1	1	0.0	.icono-fondo	Line 74:1
0.007	0	1	1	0.0	.margen	Line 170:1,Line 3...

Imagen 2.a - Captura de la tabla "Selector stats" con 0.358 ms

Summary	Bottom-up	Call tree	Event log	Selector stats		
Elapsed (...)	Invalidation co...	Match at...	Match co...	% of slo...	Selector	Style Sheet
0.333	0	306	187	58.0	(Totals for all selectors)	
0.033	0	5	4	0.0	section[class] div:first-child img	Line 161:1
0.029	0	51	49	100.0	*	Line 3:1
0.024	0	3	1	0.0	footer p	Line 137:1,Line ...
0.022	0	1	1	0.0	button	Line 65:1
0.020	0	1	1	0.0	input	Line 56:1
0.016	0	2	2	0.0	body	Line 13:1
0.013	0	1	1	0.0	.ficha-usuario	Line 225:1,Line ...
0.012	0	1	1	0.0	nav:first-child	Line 20:1
0.011	0	1	1	0.0	.margen	Line 170:1,Line ...
0.011	0	4	2	0.0	section h1	Line 35:1,Line 1...
0.009	0	2	1	0.0	section[class] a	Line 188:1,Line ...

Imagen 2.b - Captura de la tabla "Selector stats" con 0.333 ms

En el estado inicial, los selectores anidados consumían más recursos, 0.358 ms en total. Al reemplazar selectores descendentes largos por identificadores más directos, el tiempo total de emparejamiento bajó a 0.333 ms. Esto demuestra que evitar el anidamiento profundo reduce el costo computacional y mejora la velocidad de renderizado de la página.

3. Simulación de CPU Móvil y Análisis de Rendimiento

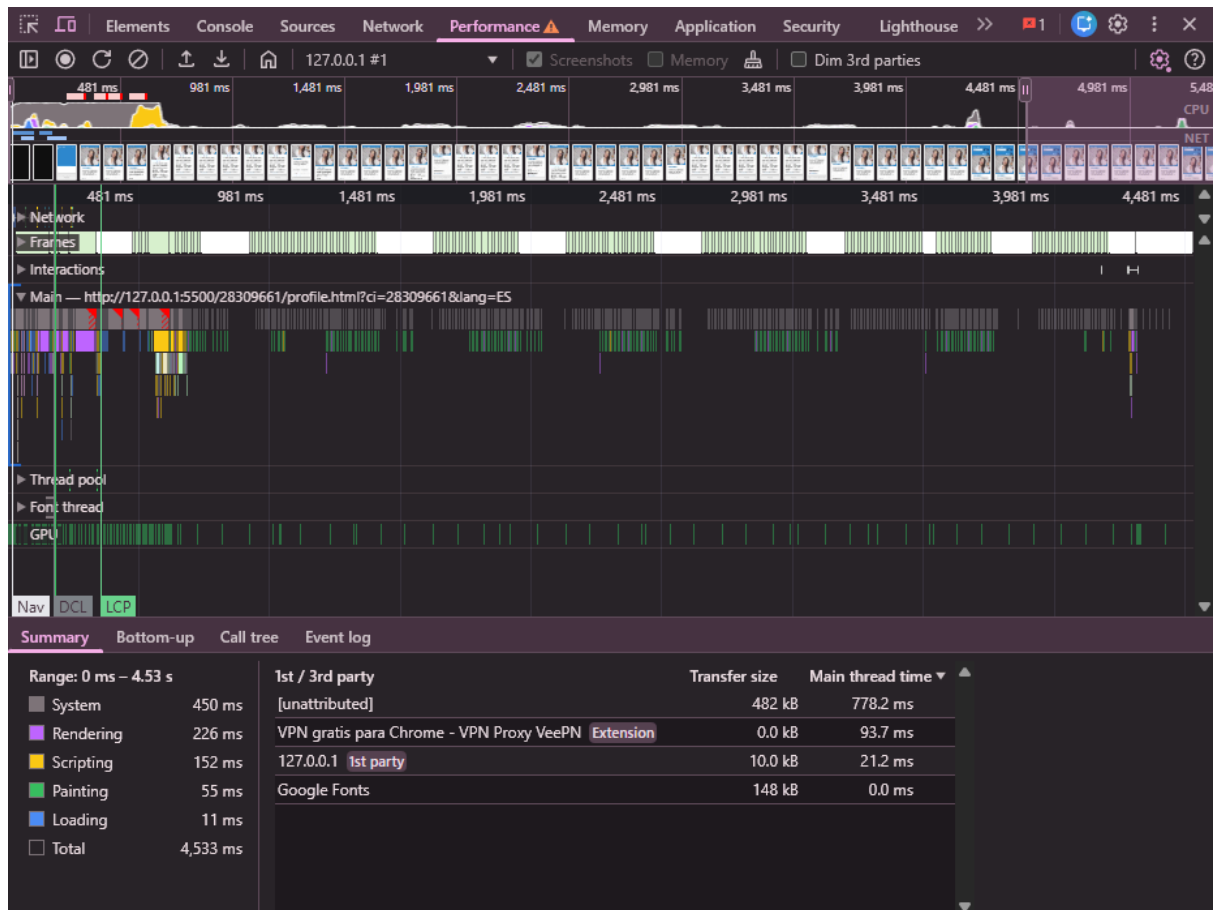


Imagen 3.a - Captura de la pestaña Performance mostrando 55 ms de Painting

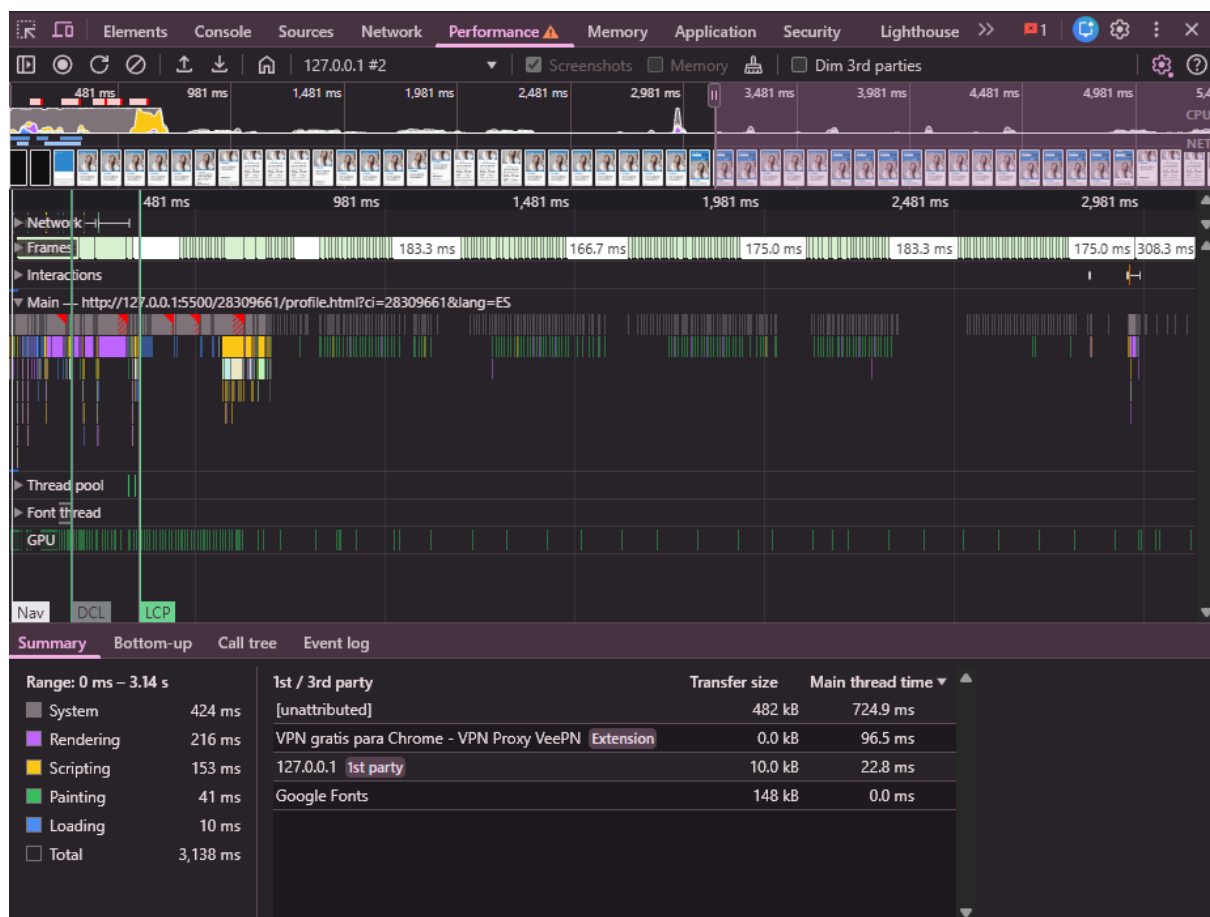


Imagen 3.b - Captura de la pestaña Performance mostrando 41 ms de Painting

Al simular un CPU móvil con rendimiento reducido (4x slowdown), analicé las métricas y detecté consumo en las tareas de repintado de estilos complejos. Para optimizarlo, apliqué `will-change: transform` y `transform: translateZ(0)` en los contenedores principales para forzar la aceleración por hardware (GPU). Al realizar la segunda prueba, se evidenció una mejora real en el hilo principal, logrando reducir el tiempo de Pintura (Painting) de 55 ms a 41 ms.